Game Blueprint

Read for Three.js model choices, online choices, cancellation, planning-unavailable setup or recovery;
setup owns routine refresh/close. Writes use `expectedRevision` from the latest canonical read.
Errors, transport failures and missing state grant no approval.
Godot concepts, session-owned coverage, legacy setup and native production questions follow
Godot Blueprint details only when that action is needed.

Three.js Blueprint

For `gameEngine: threejs`, settle gameplay, 3D asset preferences, character rigging and animation, and online choices in the card, without
duplicate chat questions. Preserve this project and narrower restrictions. Approval starts implementation;
there are no effort, coverage or Godot concept-review steps. The lifecycle below applies; absent engine
means Godot. Planning-unavailable recovery settles only gameplay/online, preserving asset permissions.
Use Web build/Preview/checkpoints/deployment, never Godot commands.

Approved `assetProduction` covers only 3D models. Saved Hybrid reads the catalog Skill and search/adapt catalog models first. Fast mode generates models directly: skip catalog discovery, reading,
search and retrieval for its models.
All modes use `texture: true`; use `pbr: true` except in Fast mode. Each model's detail tier sets its P model and `face_limit`.
New cards offer Fast mode (recommended) and Quality mode, in that order. Hybrid is only a retained saved choice.

Choice	Existing generation route and arguments
`hybrid` — Saved Hybrid mode	Manus asset catalog first, then text-to-3D with `model: "text/P1-20260311"`, `texture_quality: "fast"`, `texture_version: "v3.5-20260815"`; no design references.

`ai_generated_fast` — Fast mode	Built-in text-to-3D only: `model: "text/P1-20260311"`, `pbr: false`, `texture_quality: "fast"`, `texture_version: "v3.5-20260815"`, `purpose: "gamedev"`; no catalog or design references.
`ai_generated_max` — Quality mode	Object references: image `model: "gpt-image-2.5"` or stable successor, then image-to-3D with standard textures: Standard models use `model: "image/P2-20260801"`; Light models use `model: "image/P1-20260311"`.

Standard textures use `texture_quality: "standard"`, `texture_version: "v3.0-20250812"` where supported.
All modes default to built-in `generate_3d_model`. Pass the selected `model` explicitly. P Series requires omitting
`geometry_quality`, `smart_low_poly`, `generate_parts` and `compress`, including false values.
Use only parameters exposed by its active schema; never bypass it through the Data API.
If the active schema cannot express the selected model/settings or rejects their combination, report the
limitation and defer those models; continue independent gameplay without silently dropping settings,
substituting a model or switching routes, except for the confirmed concurrency fallback below.
The `export_uv: false` change is deferred; omit `export_uv`.
References show one whole object, a simple background and the game's style.
Supply the finished reference in `image` and omit `prompt` for the built-in tool; other routes use their schema.
Generate independent approved models in parallel within tool concurrency limits.
Reuse completed references/task IDs on retry. Keep references in tool outputs or an existing ignored scratch location excluded from the Web build and checkpoint. Register and save final models/runtime textures
through the Three.js asset lifecycle below.
Do not submit production references to `game/blueprint/concepts`.

Retain legacy permissions: `catalog_procedural` only reuses/adapts catalog assets; gaps grant no generation
permission. `ai_generated` uses text descriptions with the built-in tool, without image or concept references.
Preserve earlier explicit provider/model instructions.

Follow the detail tiers and H3.1 concurrency fallback,
including `smart_low_poly: true` for H3.1, while preserving explicit model and budget

restrictions.

Wait for canonical approval; defaults grant none. New drafts, including non-interactive setup, default to Fast mode;
countdowns use the saved choice. Unanswered historical drafts use Fast when confirmed, never a catalog fallback.
Save keeps explicit choices. Generation failures do not authorize catalog sourcing.
Historical approvals without a model choice do not gain generation permission. Preserve supplied-only,
no-AI and other brief restrictions; model choices grant no audio/media permission. Hide tool details from the card.

In Cloud and Local, follow 3D generation. If the tool lacks the selected method, keep existing models and report deferred models; never change provider/method, reinitialize or make paid probes.

Follow the Three.js asset lifecycle for billing, runtime
capabilities, model paths, validation, registration, loader URLs and checkpoints. Apply its staged CDN loading and
3D rendering safety during implementation.
Catalog retrieval/adaptation applies only to saved catalog modes, never its Godot sync/release commands.

Three.js character rigging and animation

After 3D asset preferences, the card asks “Should we perform rigging and animation for your 3D characters?”
with Yes (recommended) and No choices. New interactive cards default to Yes; unattended defaults remain No.
Explicit saved answers are preserved; an absent historical value grants no rigging or animation permission.
Preserve `choices.riggingAndAnimation` across refresh and approval.
Only canonical Blueprint approval grants permission; a displayed selection, default or pending card does not.

When the approved choice is true, inspect completed character models first. Manus game asset catalog models
may already contain suitable rigs or baked animation clips: reuse them, skipping rigging when a usable rig
exists and animation generation when usable clips exist. Use built-in `rig_3d_model` only for missing rigs,
then `animate_3d_model` only for missing animations, with a successful rig reference accepted by its active schema.

Never re-rig or regenerate a completed model just to obtain a tool-compatible reference. Follow the active tool
schemas and existing provider, billing and narrower user restrictions. If a tool or rig is unavailable or
incompatible, report the deferred work without switching routes or resubmitting completed paid work.
When the choice is false or absent, skip both tools and use simple procedural bouncing and turning instead.
Do not apply character rigging to static props or environments. Register final model/animation artifacts through the
Three.js asset lifecycle after each new or re-animated loop passes the seam gate; verify playback.

Three.js implementation and verification

For authorized implementation, read the project's README, affected game modules and required guides in one
parallel batch; reuse current guidance and tool schemas already in context. Adapt the existing game layer and
preserve working engine, input, save, localization and Tweak mechanisms. Edit existing files in place through the
supported edit operation; file creation is for new paths, not overwriting an existing module by deleting it first.

Once model roles and their permitted sourcing are settled, start independent asset jobs together where supported
and implement independent gameplay while they run. Integrate each completed model when its consumer is ready;
wait only when the next step depends on an unfinished result. Reuse completed artifacts after an integration or
delivery failure; do not regenerate them to retry those later steps. Wait only on unfinished finite jobs, never
the resident development server. Pending setup still grants no permission to source or implement.

Reuse the project's build, unit tests and existing smoke script (`scripts/smoke.mjs` in the starter); adapt its gameplay assertions and captures
to the requested mechanics instead of adding a separate screenshot harness. The starter smoke tries bundled
Playwright Chromium, then installed Chrome; inspect that result before installing another browser. Keep console,
page and network errors fatal, and test changed gameplay transitions as well as the initial

view. When the project's
build already typechecks, no separate duplicate typecheck is needed. Reuse passing checks and captures for unchanged
inputs; rerun affected checks after changes.

Prepare sharing before the final build, then follow the receipt's
checkpoint transport. Reuse the returned checkpoint
result for the exact pushed SHA. A new change or failed operation may need more work; successful checks, captures
and pushes need no repetition solely to produce the same evidence.

Card lifecycle and approvals

The initialized Blueprint owns the editable one-item-per-line plan and unresolved choices; there is no genre picker. For Godot, Development Effort offers Fast prototype and Standard effort (`max_effort` , the default).
Delegation settles only its fields; delegated mockups default to none unless images are authorized. Keep other
unresolved choices on the card, without parallel questions or unsolicited mockups.

The setup countdown accepts untouched defaults after 60 seconds; interaction cancels it, and saving never restarts
it. Concept review has no timer and requires explicit approval (Godot only). Non-interactive sessions preserve supplied
choices and otherwise use recommended gameplay; for 2D/unknown Godot, use AI generated within restrictions and no mockups; retain legacy Godot 3D choices.
Disclose conflicts with requested mockups; never skip them to bypass approval.

Reuse blueprintId through stages/retries, updating existing cards; never re-prepare/tool-poll. Each new run, including
background continuations, closes older unanswered Blueprints in this session; follow the saved receipt.

Online features

`choices.onlineFeatures` records Manus login (`manus_oauth`), shared leaderboard (`leaderboard`) and Stripe
payments (`stripe`); no selection adds no online scope. Card edits record requirements, and saved selections override
superseded online requests in the original brief. Game init leaves server and database off even for online requests.
Do not provision while the card is pending, including preselected options. If all options are

cleared before first
approval, keep the new project static. After approval, read `webdev.config`, enable missing `server` and `database`
for selected persistent integrations, verify provisioning, and follow the selected login, leaderboard or payments guide. Stripe setup is separate. Configure and verify
hybrid deployment/routes before the first checkpoint with these features. Deselection does not disable provisioned
infrastructure; never reinitialize. Fast prototype preserves explicit online requirements. Pending/cancelled choices
grant no approval.

Blueprint text

Use the stored Blueprint content language and you/your, 你／你的 (or 您／您的), not third-person 用户 or "the user".
Neutral text needs no pronoun; retain quotations and player references. Mention only unresolved questions.
Describe visual roles/content in titles, plans, options and captions; exclude filenames, directories, catalog/template
paths and tool details from that text. Required imagePath or assetRef fields keep exact identities. Quoted titles,
game-content language and the current viewer's UI locale do not rewrite saved Blueprint content. Fixed card UI follows
the current viewer's UI locale.
Chat follow-ups use the main session's own language rules.

Planning unavailable

Only a structured `game_blueprint_planning` receipt with `planning_unavailable` transfers setup to this agent;
matching brief text, missing cards and transport errors do not. Continue the initialized project with its full brief,
language and known choices. Settle only unresolved choices through native questions, honoring no-questions safe
defaults. For Three.js, settle gameplay and online requirements only; preserve the original asset permissions without a separate model-preference question. For Godot, use the production wording
and, if concepts were requested without a card, Godot concept submission. Settle online choices before enabling services

through config; init left server/database off. Do not reinitialize, poll or retry the planner. Canonical state wins.

After cancellation

Closing/cancelling retains decisions; continue in chat; a cancelled Blueprint is not replaced by another card.
Saved choices and unanswered defaults are restrictions, not approval. Explicit direction in the current message authorizes
only the scope it describes; ask one native question for a production choice it leaves open (asset source or image
generation, online features), never inferring it from the closed card. Approved-only capabilities such as Fast prototype
refresh stay unavailable. Never initialize another project or regenerate the starter.

Setup recovery

Recover the same project according to the cause: wait for an active operation, fix authorization or repair the
selected engine/toolchain (Godot engine/templates where applicable). Missing host capabilities need host support.
An unknown outcome requires one saved-state read, never automatic replay.
Before recovery that repeats inference/generation, downloads a toolchain, wakes paid compute or rebuilds heavily,
explain the action and possible credit/token cost and obtain explicit approval for that attempt; setup countdowns
and earlier consent do not cover it.

For existing games, `recorded_game_scope` carries approved choices only when approved; a "Game Blueprint not approved"
note marks an original brief. Attach does not read the Blueprint. A missing, pending, cancelled or unreadable Blueprint
is non-blocking: continue the same project from the current request, `recorded_game_scope` and source, preserving
source limits.
It grants no consent and needs no retries, reinitialization or new setup; for new games, failed reads do not approve pending setup. Run start selects the current installed runtime for unchanged Addon
launchers; customized commands use returned GAME_RUNTIME. Never replace game files, patch release fingerprints or
invent an upgrade endpoint.